

TP — Délices de Douala

La couche de données : services, signals, HTTP & environnements

Contexte

Au TP précédent, vous avez construit l'interface de « Délices de Douala », un restaurant de Douala : la carte affichait des plats écrits en dur dans le composant. Ça suffisait pour l'affichage, mais ce n'est pas viable : impossible de partager ces données entre pages, ni de les mettre à jour proprement.

Votre mission : transformer cette carte statique en une vraie couche de données Angular moderne — centralisée dans un service, réactive, alimentée depuis un « serveur », et configurable par environnement.

Objectifs

- Centraliser l'état du menu dans un service avec des signals.
- Injecter le service avec `inject()` et afficher la carte avec `@if / @for`.
- Charger le menu depuis un fichier JSON via `httpResource()` et gérer les 3 états.
- Filtrer par catégorie avec un signal + un `computed`.
- Afficher un « plat du jour » qui tourne, via un `Observable` `RxJS` converti en signal.
- Externaliser la configuration dans les variables d'environnement.

Mise en place

Repartez de votre application « Délices de Douala ». Lancez le serveur de développement :

```
ng serve --host 0.0.0.0 --port 8080
```

Créez le fichier de données statiques ci-dessous (votre « serveur » local) dans `public/api/plats.json` — il est servi à l'URL `/api/plats.json` :

```
// public/api/plats.json (servi à /api/plats.json)
[
  { "id": "d1", "nom": "Ndolè aux crevettes", "prix": 3500, "categorie": "Plats", "disponible": true },
  { "id": "d2", "nom": "Poulet DG", "prix": 4000, "categorie": "Plats", "disponible": true },
  { "id": "d3", "nom": "Poisson braisé + miondo", "prix": 3000, "categorie": "Grillades", "disponible": true },
  { "id": "d4", "nom": "Eru + water fufu", "prix": 2500, "categorie": "Plats", "disponible": true },
  { "id": "d5", "nom": "Koki + plantain mûr", "prix": 2000, "categorie": "Végétarien", "disponible": true },
  { "id": "d6", "nom": "Jus de bissap", "prix": 1000, "categorie": "Boissons", "disponible": true },
  { "id": "d7", "nom": "Jus de gingembre", "prix": 1000, "categorie": "Boissons", "disponible": false }
]
```

Missions

Mission 1 — Le modèle et le service

Définissez une interface `Plat` (`id`, `nom`, `prix` en FCFA, `categorie`, `disponible`). Générez un `MenuService` (`@Injectable` `providedIn root`). L'état du menu doit être encapsulé (signal privé) et exposé en lecture seule ; aucune propriété mutable publique.

✓ Attendu : `signal()` + `asReadonly()` (ou `httpResource`, voir Mission 3) ; méthodes de mutation via `update()/set()`.

Mission 2 — Injection et affichage réactif

Dans un composant `CarteComponent` (standalone), injectez le service avec `inject()`. Affichez la liste des plats avec `@for (...; track ...)` et un `@if` pour le cas « liste vide ». Les prix s’affichent en FCFA :

```
<!-- Affichage d'un prix en FCFA -->
{{ plat.prix | currency:'XAF': 'symbol': '1.0-0' }}
```

✓ Interdit : injection par constructeur, `ngOnInit` pour charger les données, `*ngIf` / `*ngFor`.

Mission 3 — Le menu vient du serveur

Le menu ne doit plus être écrit en dur : chargez-le depuis `/api/plats.json` avec `httpResource()`. Le template doit gérer les trois états : chargement (`isLoading`), erreur (`error`), et données (`value`). Activez d’abord `provideHttpClient()` dans `app.config.ts`.

✓ Attendu : un message pendant le chargement, un message en cas d’erreur, la carte sinon.

Mission 4 — Filtre par catégorie

Ajoutez des boutons de catégorie (Toutes, Plats, Grillades, Végétarien, Boissons). La catégorie sélectionnée est un signal ; la liste affichée est un computed dérivé du menu et de la catégorie. Aucun recalcul manuel.

Mission 5 — Le plat du jour

Affichez un « plat du jour » tiré du menu, qui change automatiquement toutes les 5 secondes. Utilisez un `Observable RxJS (interval)` converti en signal avec `toSignal()` — donc sans `subscribe` manuel ni `ngOnDestroy`.

Mission 6 — Variables d’environnement

Générez les fichiers d’environnement (`ng generate environments`). Externalisez l’URL du serveur (`serverUrl`) et le nom du restaurant. Le service utilise `environment.serverUrl`. Import toujours depuis le fichier de base — jamais `.development` ni `.prod`.

Bonus (facultatif)

- Recherche par nom : un input signal relié à un `httpResource` paramétré (ou un computed de filtrage).
- Griser les plats indisponibles (`disponible: false`) et afficher un badge « épuisé ».

Contraintes techniques — charte ATL2026

Obligatoire

- Composants standalone · signals pour l'état · @if / @for.
- inject() pour l'injection · interfaces TypeScript pour vos modèles.
- provideHttpClient() dans app.config.ts · httpResource() pour les lectures.
- toSignal() / takeUntilDestroyed() pour les Observables.
- Serveur en port 8080, --host 0.0.0.0 · commits conventionnels.

Interdit — pénalités automatiques (-2 pts chacun)

- Injection par constructeur · ngOnInit pour initialiser des données.
- *ngIf / *ngFor · @Input / @Output décorateurs · EventEmitter.
- HttpClientModule · subscribe manuel sans nettoyage.
- Données du menu écrites en dur dans le composant.

Livrables & modalités

- Travail en binôme.
- Dépôt GitHub (branches main / formateur / apprenant), commits conventionnels réguliers.
- Application déployée sur Vercel et fonctionnelle.
- Soumission du lien Vercel via le formulaire de suivi de la cohorte.

Barème

Critère	Attendu	Points
Modèle & service	Interface Plat ; MenuService providedIn root ; état encapsulé (signal privé + asReadOnly).	8
Injection & affichage	inject() ; @if / @for avec track ; prix en FCFA (currency:'XAF').	6
Menu depuis le serveur	public/api/plats.json ; httpResource() ; 3 états chargement / erreur / données.	12
Filtre par catégorie	signal « categorie » + computed « platsFiltres » ; boutons fonctionnels.	6
Plat du jour	interval(5000) → toSignal ; rotation automatique, sans subscribe manuel.	6
Environnement	environment.ts + environment.development.ts ; import correct ; serverUrl utilisé.	6
Charte ATL2026	Standalone, interfaces TS, aucun pattern déprécié.	4
Livraison	Déploiement Vercel fonctionnel + commits conventionnels.	2
TOTAL		/ 50

Rappel : chaque pattern déprécié détecté retire 2 points (plafonné). Le bonus peut compenser une pénalité, sans dépasser 50.

Conseils & pièges

- Signal lu dans le template : n'oubliez pas les parenthèses — menu.value(), plats(), categorie().
- @for exige toujours un track (track plat.id).
- Erreur 404 au chargement ? Vérifiez le chemin exact du fichier (plats.json) et l'URL du serveur.
- httpResource sert à lire (GET) ; pour une écriture, passez par HttpClient.